

Sistemas en Tiempo Real

3º Curso – Grado en Ingeniería Informática
Especialidad Ingeniería de Computadores



Sistemas en Tiempo Real

Programa de Teoría

Tema 1: Introducción a los Sistemas en Tiempo Real (2 h.)

Tema 2: Lenguajes para aplicaciones en Tiempo Real (2 h.)

Tema 3: Interfaces y Elementos Hardware (4 h.)

Tema 4: Concurrencia y Sincronización (6 h.)

Tema 5: Sistemas Operativos en Tiempo Real (6 h.)

Tema 6: Planificación de Tiempo Real (6 h.)



Tema 2. Lenguajes para Aplicaciones en Tiempo Real

- 
1. Características deseables en los lenguajes para aplicaciones en Tiempo Real
 1. Seguridad, Legibilidad, Flexibilidad, Portabilidad
 2. Simplicidad, Modularidad, Eficacia
 2. ADA como lenguaje para Tiempo Real
 3. Tratamiento de los errores de ejecución
 4. Entorno de ejecución de la aplicación



Tema 2. Lenguajes para Aplicaciones en Tiempo Real

- ¿Qué lenguaje se puede utilizar para programar aplicaciones de Tiempo Real?
 - CUALQUIERA
 - Determinados lenguajes facilitan la programación de aplicaciones de Tiempo Real:
 - Hay lenguajes específicamente diseñados para dar soporte concurrente a las tareas: ADA, Occam, Java-RT
 - Hay lenguajes que consiguen la concurrencia mediante extensiones del lenguaje: C/C++

Ensamblador cuesta mucho concurrencia



Tema 2. Lenguajes para Aplicaciones en Tiempo Real

• Características deseables en los lenguajes para aplicaciones en Tiempo Real

• Seguridad

- Efectividad del compilador y del sistema en ejecución para detectar automáticamente errores de programación.

• Características que lo facilitan:

- Datos tipificados
- Declaración previa de todas las variables
- Sintaxis no ambigua

Python no



Tema 2. Lenguajes para Aplicaciones en Tiempo Real

- Características deseables en los lenguajes para aplicaciones en Tiempo Real
 - Legibilidad
 - Facilidad para entender el programa sin necesidad de documentación externa adicional.
 - Habitualmente se consigue:
 - Uso de comentarios descriptivos
 - Pocas restricciones en la nomenclatura de las variables y funciones

Tema 2. Lenguajes para Aplicaciones en Tiempo Real

- Características deseables en los lenguajes para aplicaciones en Tiempo Real
 - Flexibilidad
 - Capacidad de un lenguaje para describir todo tipo de funcionalidad sin necesidad de recurrir a otros lenguajes o extensiones del lenguaje (habitualmente, a preprocesadores, librerías o al ensamblador).
 - Especialmente:
 - Acceso al hardware
 - Control de procesos y tareas
 - Control de recursos



Tema 2. Lenguajes para Aplicaciones en Tiempo Real

- Características deseables en los lenguajes para aplicaciones en Tiempo Real
 - Portabilidad
 - Mide la capacidad que tiene un lenguaje para ejecutar el mismo código en diferentes máquinas.
 - No es tan importante como para aplicaciones “estándar”, ya que se asume que existen restricciones con respecto al hardware en el que se ejecutan las aplicaciones de TR.
 - En particular,
 - Problemas con tamaño de registros
 - Problemas con la precisión en la representación numérica



Tema 2. Lenguajes para Aplicaciones en Tiempo Real

- Características deseables en los lenguajes para aplicaciones en Tiempo Real
 - Simplicidad
 - Mide la capacidad de un lenguaje para describir con pocas sentencias de baja complejidad las diferentes funciones que se desean que ejecute una aplicación.
 - La simplicidad reduce la probabilidad de errores, reduce el costo y aumenta la seguridad y la portabilidad.

Tema 2. Lenguajes para Aplicaciones en Tiempo Real

- Características deseables en los lenguajes para aplicaciones en Tiempo Real
 - Modularidad
 - Mide la capacidad de un lenguaje para definir subrutinas dentro de su código de manera que estas subrutinas se puedan desarrollar como módulos independientes capaces de interactuar entre sí.
 - Imprescindible:
 - Control estricto de los parámetros de entrada y salida
 - Ocultación de la información de un módulo a los demás
 - Especificación clara y precisa tanto de la función que realizan como de la interfaz
 - Posibilidad de compilación y depuración separada de cada módulo



Tema 2. Lenguajes para Aplicaciones en Tiempo Real

- Características deseables en los lenguajes para aplicaciones en Tiempo Real
 - Eficacia
 - Mide la capacidad de los lenguajes para garantizar una ejecución rápida con un rendimiento garantizado que sea capaz de cumplir con las restricciones temporales.
 - Énfasis en:
 - Tamaño del código
 - Velocidad de ejecución
 - Uniformidad en los tiempos de ejecución
 - Deben permitir la medición exacta de los tiempos de ejecución de cada tarea.
 - Dificultad en la predicción del tiempo de ejecución:
 - Variables que varían dinámicamente su tamaño (cadenas, etc.)
 - Bucles tipo **while**.



Tema 2. Lenguajes para Aplicaciones en Tiempo Real

1. Características deseables en los lenguajes para aplicaciones en Tiempo Real
- ➔ 2. ADA como lenguaje para Tiempo Real
 1. Introducción a ADA
 2. Léxico
 3. Tipos de Datos
 4. Instrucciones
 5. Subprogramas
 6. Estructura de programas
3. Tratamiento de los errores de ejecución
4. Entorno de ejecución de la aplicación



Tema 2. Lenguajes para Aplicaciones en Tiempo Real

• Introducción a ADA

- ADA es un lenguaje de programación derivado de Pascal diseñado específicamente para la creación de aplicaciones de "larga duración".
- Desarrollado por iniciativa del Departamento de Defensa (DoD) de los EEUU para desarrollar sistemas empotrados y de Tiempo Real.
 - Cualquier desarrollo con el DoD sobre STR tiene que estar especificado en ADA.
- Actualmente se utiliza la versión ADA95.
- GNU tiene un compilador para ADA: *gnat*
 - <http://www.gnu.org/software/gnat/gnat.html>



Tema 2. Lenguajes para Aplicaciones en Tiempo Real

• Léxico

- Los **identificadores** pueden contener letras mayúsculas y minúsculas, dígitos o el guión bajo `\_`.
 - Sensor, tiempo_De_activacion.
- En ADA no se distingue entre mayúsculas y minúsculas.
- Las **palabras reservadas** no pueden ser utilizadas como identificadores.
- Se puede usar el carácter separador con los números para claridad:
 - 123, 150_000 (=150000), 3.141_592_654 (=3.141592654), 1.7E-5
- Se usa el carácter `#` para indicar un número en otra base.
 - 2#10001100# (En base 2, 10001100 = 140 en Decimal)
- Los comentarios empiezan por `--` y van hasta final de línea.



Tema 2. Lenguajes para Aplicaciones en Tiempo Real

`int a = 1`

`float b = 0.5`

C → es tipificado débil (no bueno)

- **Tipificación de Datos (general)** $(int)a + (int)b > 1.5$.
 - Tipificación fuerte de datos
 - Todos los datos tienen asociado un tipo de dato.
 - Solo se pueden realizar operaciones entre datos del mismo tipo. No se realizan conversiones de tipo, salvo que se realicen de manera explícita.
 - Los resultados dan siempre un resultado del tipo correcto.
 - Tipificación débil de datos
 - Los datos pueden no tener tipo de dato asociado.
 - Se realizan conversiones de tipo de manera automática entre tipos de datos "compatibles".
 - Los resultados pueden tener un tipo de dato no coincidente con las entradas.
 - Problemas con lenguajes con tipificación débil: FORTRAN



Tema 2. Lenguajes para Aplicaciones en Tiempo Real

- Tipificación de Datos (general)
 - Precisión de los datos numéricos
 - Variables con el mismo tipo pueden tener precisiones diferentes en función de la arquitectura:
 - Enteros: 8 bits, 16 bits, 32 bits, 64 bits.
 - Reales: 16 bits, 32 bits, 64 bits, 80 bits, 128 bits.
 - El rango de trabajo de las variables y la precisión entre números determinan el número de bits.



Tema 2. Lenguajes para Aplicaciones en Tiempo Real

• Tipos de Datos (ADA)

- ADA presenta **tipificación fuerte**.
- Tipos enumerados:
 - *Boolean*
 - *Character*
 - Definido por el usuario:
 - ***type*** *Modo* ***is*** (*Rapido*, *Lento*);
- Números enteros:
 - Con signo:
 - *Integer*
 - ***type*** *Indice* ***is range*** 1 .. 10;
 - Sin signo (modulares)
 - ***type*** *Octeto* ***is mod*** 256;



Tema 2. Lenguajes para Aplicaciones en Tiempo Real

Tipos de Datos (ADA)

Números reales: (Coma flotante)

- Se especifican el número de dígitos con los que se representará el número real, pero no se indica nada sobre la precisión ni el grado de exactitud sobre el rango.
- *Float* -- Reales con 6 dígitos de precisión.
- **type** *Temperatura* **is digits 5 range** 0.0 .. 100.0;
- *digits* indica el número mínimo de dígitos para representar el rango.

Números reales: (Coma fija)

- Ordinarios: Se especifica el grado de exactitud que ha de mantenerse durante todo el rango del tipo.
 - *Duration* -- Se utiliza para control de tiempo. Precisión: 1E-9
 - **type** *Punto_Fijo* **is delta 0.1 range** -1.5 .. 1.0;
- Decimales: Se especifica el grado de exactitud y el número de dígitos de precisión que ha de tener:
 - **type** *Decimal* **is delta 0.01 digits 5**;
 - -999.99 .. +999.99



Tema 2. Lenguajes para Aplicaciones en Tiempo Real

Tipificación y Uso de Datos (ADA)

Definición:

nombreVariable* : Tipo;

Asignación:

nombreVariable := valor;

Ejemplo:

```

type Indice is range 1..100;
type Longitud is digits 5 range 0..100.0;
first, last: Indice;
front, side: Longitud;

last := first + 15;           -- correcto
side := 2.0*front;           -- correcto
side := 2*front;             -- INCORRECTO
side := front + 2.0*first;    -- INCORRECTO
side := front + 2.0*Longitud(first); -- correcto
    
```



Tema 2. Lenguajes para Aplicaciones en Tiempo Real

Tipos Compuestos

- *String*

- *Array*

- ***type* Temperaturas *is array* (Indice) *of* Temperatura;**

- Los tipos Temperatura e Indice deben estar previamente definidos

- ***type* matriz *is array* (1..10, 1..10) *of* Float;**

- Ejemplos:

```
t: Temperaturas;
```

```
t(5) := 1.0;
```

```
t := (25.0, 23.0, 23.0, 23.0, 20.0, 23.0, 23.0);
```

```
t := (1=>25.0, 5=>20.0, others=>23.0);
```



Tema 2. Lenguajes para Aplicaciones en Tiempo Real

Tipos Compuestos

- Registros: Tipo de dato compuesto que permite agrupar diferentes tipos de datos en un tipo de dato abstracto.

- ***type* TipoRegistro *is record***

- *variable: Tipo;*

- ***end record;***

- Ejemplo:

```
type Operacion is record
  m : Modo;
  temp : temperatura;
end record;
```

```
Op: Operacion;
```

```
Op.m := Rapido;
```

```
Op.temp := 25.0;
```

```
Op := (Rapido, 25.0);
```

```
Op := (m => Rapido, temp => 25.0);
```



Tema 2. Lenguajes para Aplicaciones en Tiempo Real

• Tipos Acceso (Punteros)

- Punteros: Designan valores de otros tipos.

- **type** operacionAcceso **is access** Operacion;

- Se pueden acceder a objetos creados dinámicamente:

- *t*: operacionAcceso := **new** Operacion;

- Si no se indica nada, los nuevos objetos se inicializan a **null**.

- Acceso dinámico:

```
t.m := Lento;
```

```
t.all := (m => Lento, temp => 20.0);
```

```
t := new Operacion' (m=>Lento, temp => 20.0);
```

Tema 2. Lenguajes para Aplicaciones en Tiempo Real

• Instrucciones

• Simples:

- Asignación: *variable := valor;*
- Llamada a procedimiento: *funcion(parametros);*

• Compuestas:

• Bloque:

• **declare**

- A: integer; -- Parte de declaraciones de variables

• **begin**

- Leer(a);
- Imprimir(a);

• **end;**



Tema 2. Lenguajes para Aplicaciones en Tiempo Real

Instrucciones

Selección:

- if** $a > b$ **then**

- $max := a;$

- elsif** $a < b$ **then**

- $max := b;$

- else**

- $max := 0;$

- end if;**

Selección por casos:

- Case** *dia* **is**

- when** *lunes*

$\Rightarrow principio_semana;$

- when** *martes..viernes*

$\Rightarrow mitad_semana;$

- when** *sabado|domingo*

$\Rightarrow fin_semana;$

- when** *others*

$\Rightarrow no_hay;$

- end case;**

- Se realiza sobre una variable discreta y hay que cubrir todas las alternativas de la variable.



Tema 2. Lenguajes para Aplicaciones en Tiempo Real

• Instrucciones

• Iteración:

• Bucle finito:

- **for** *i* **in** 1..10 **loop**

- leer(*v(i)*);

- **end loop;**

- La definición de *i* es implícita en función del rango.

• Bucle condicional:

- **while** *a* < *b* **loop**

- *a*:=*a*+1;

- **end loop;**

• Bucle condicional final:

- **loop**

- leer(*d*);

- **exit when** *d*=0;

- **end loop;**



Tema 2. Lenguajes para Aplicaciones en Tiempo Real

• Subprogramas

- La especificación se declara en una zona declarativa.
- Procedimientos sin parámetros:
 - **procedure** nombreProc;
- Procedimientos con parámetros:
 - **procedure** proc (param1: **in out** Tipo; param2 : **in** Tipo:= ValorDefecto);
- Funciones:
 - **function** nombreFuncion (param1: TipoA; param2 : TipoB) **return** TipoC;
- Tres modos de los parámetros:
 - **In**: No se modifica el valor al ejecutar el subprograma. Puede tomar un valor por defecto.
 - **Out**: El valor del parámetro será asignado por el subprograma.
 - **In out**: el valor del parámetro es tanto de entrada como de salida.
- Las funciones solo pueden tomar parámetros de tipo **in**.



Tema 2. Lenguajes para Aplicaciones en Tiempo Real

• Subprogramas

- El cuerpo del subprograma se coloca tras la zona declarativa.

• Procedimientos:

- **procedure** Incremento (param1: **in out** integer; param2 : **in** integer:= 1) **is**
- **begin**
 - param1 := param1 + param2;
- **end** Incremento;

• Funciones

- **function** maximo (a,b: integer) **return** integer **is**
- **begin**
 - **if** a>b **then return** a;
 - **else return** b;
 - **end if**;
- **end** maximo;



Tema 2. Lenguajes para Aplicaciones en Tiempo Real

• Estructura de programas

- Un programa en ADA está formado por *unidades de programa*.
- Distintas unidades de programa:
 - Subprogramas: Funciones y procedimientos.
 - Paquetes: Unidad lógica para definir una colección de entidades lógicamente relacionadas.
 - Otras: Tareas, Objetos protegidos, Unidades genéricas.
- Las unidades de programa tienen 2 partes: declaración y cuerpo.
- Un programa en ADA está formado por:
 - Un procedimiento principal.
 - Otros subprogramas o paquetes escritos por el programador.
 - Subprogramas o paquetes precompilados.



Tema 2. Lenguajes para Aplicaciones en Tiempo Real

1. Características deseables en los lenguajes para aplicaciones en Tiempo Real
2. ADA como lenguaje para Tiempo Real
- 3. Tratamiento de los errores de ejecución
 1. Errores de compilación y de ejecución
 2. Excepciones. Reanudación o terminación tras error
3. Entorno de ejecución de la aplicación



Tema 2. Lenguajes para Aplicaciones en Tiempo Real

- Tratamiento de los errores de ejecución
 - Tipos de Errores:
 - En Tiempo de Compilación.
 - Deben ser detectados por el compilador.
 - En Tiempo de Ejecución (Excepciones).
 - No los puede detectar el compilador y se producen durante la ejecución de la aplicación (Ej.: División por cero, logaritmo de un número negativo...)
 - En general, cuando se produce un error de ejecución se genera un mensaje por la pantalla y se aborta el programa. Esto no es aceptable en una aplicación de STR.
 - Los lenguajes con soporte para STR deben proporcionar mecanismos para detectar errores de ejecución y dar una respuesta a los mismos:
 - Simple
 - Ejecución rápida (bajo "overhead")
 - Libertad para seleccionar las excepciones ante distintas excepciones.



Tema 2. Lenguajes para Aplicaciones en Tiempo Real

- Tratamiento de los errores de ejecución
 - El tratamiento de las excepciones se limita a bloques de código: cada bloque de código deberá hacer algo diferente si se produce una determinada excepción.
 - En ADA:
 - **Declare**
 - Subtype Temperatura is Integer range 0..100;
 - **Begin**
 - -- Lectura del sensor de Temperatura y asignación del valor
 - **Exception**
 - -- Manejador
 - **End;**
 - Si un valor se sale "fuera de rango" se lanza la excepción *Constraint_Error*.
 - En C++:
 - **try** {
 - // Bloque de sentencias protegidas
 - }
 - **catch** (nombre_excepcion) {
 - // Manejador de la excepción
 - }
 - No todos los bloques pueden tener manejador de excepciones.



Tema 2. Lenguajes para Aplicaciones en Tiempo Real

• Tratamiento de los errores de ejecución

• Propagación de la excepción.

- ¿Qué hacer cuando se genera una excepción en un bloque que no posee manejador asociado?
- 1ª Solución: El compilador avisaría al programador de la ausencia del manejador => Elevada complejidad en tiempo de compilación.
- 2ª Solución: Buscar los manejadores que estén activos en la cadena de procedimientos en la pila de llamada actual: *Propagación de la excepción.*
- Para limitar la propagación en un punto y evitar que “suba” hasta el procedimiento principal se permite el uso de manejadores “catch-all” que capture todas las excepciones.



Tema 2. Lenguajes para Aplicaciones en Tiempo Real

- Tratamiento de los errores de ejecución
 - Reanudación vs. Terminación.
 - ¿Qué hacer con el procedimiento que ha producido la excepción tras la ejecución del manejador?
 - Modelo de Reanudación o de Notificación:
 - Si el manejador ha podido solucionar el problema, el procedimiento continúa.
 - Modelo de Terminación o de Escape:
 - El manejador no devuelve el control al procedimiento.
 - Modelo Híbrido:
 - En cada caso el propio manejador decidirá qué debe hacer (continuar o terminar).



Tema 2. Lenguajes para Aplicaciones en Tiempo Real

1. Características deseables en los lenguajes para aplicaciones en Tiempo Real
2. ADA como lenguaje para Tiempo Real
3. Tratamiento de los errores de ejecución
- 4. Entorno de ejecución de la aplicación
 1. Mecanismo de creación de un ejecutable
 2. Ejecutable autocontenido (stand-alone executable)
 3. Ejecutable con enlaces dinámicos (dynamically linked executable)



Tema 2. Lenguajes para Aplicaciones en Tiempo Real

- Entorno de ejecución de la aplicación
 - Mecanismo de creación de un ejecutable
 - Habitualmente, la plataforma de desarrollo suele ser diferente de la plataforma de explotación.
 - Para crear las aplicaciones, se utilizan *compiladores cruzados*.
 - La mayoría de las veces el ejecutable se introduce en el sistema de explotación de modo *off-line*: mediante cable, inalámbrico, disquete, ROM, Flash, etc.
 - Las plataformas de explotación suelen estar “vacías” (sin S.O.) por lo que, la aplicación debe incluir todas las funciones del sistema operativo, así como un pequeño kernel que proporcione una funcionalidad mínima del S.O.
 - En los STR se permite incluir solo aquellas funciones que sean necesarias, el resto no se incluyen => Se reduce el código.



Tema 2. Lenguajes para Aplicaciones en Tiempo Real

- Entorno de ejecución de la aplicación
 - Ejecutable autocontenido (stand-alone executable)
 - Se genera un único fichero ejecutable que contiene todo el código, tanto de la aplicación como de las funciones del S.O. como de las extensiones necesarias.
 - Ventajas:
 - Rapidez: Se conoce *a priori* la posición de inicio de todas las funciones.
 - Desventajas:
 - Aumento del tamaño.
 - Imposibilita la actualización de funciones del S.O. sin parar el sistema.



Tema 2. Lenguajes para Aplicaciones en Tiempo Real

- Entorno de ejecución de la aplicación
 - Ejecutable con enlaces dinámicos (dynamically linked executable)
 - Se genera un fichero que contiene solo el código de la aplicación. El resto de funciones y recursos se encuentran en otros ficheros, que son llamados dinámicamente y llevados a la memoria bajo demanda.
 - Ventaja:
 - Reducción en el tamaño del fichero de aplicación.
 - Actualización de funciones del S.O.
 - Desventajas:
 - Velocidad de ejecución.
 - Mayor consumo de memoria.
 - Control dinámico de las dependencias.



Preguntas ...

Volver

